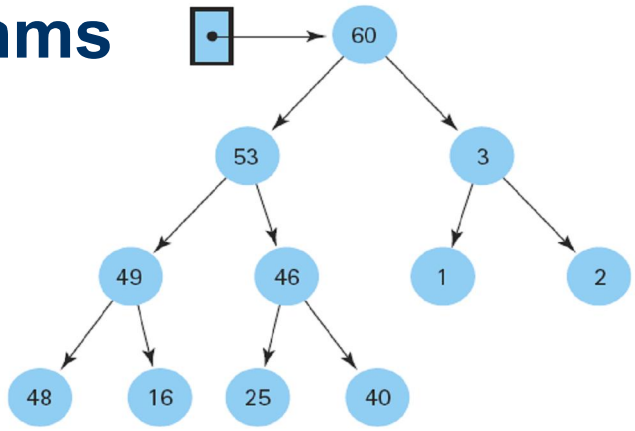
Data Structures & Algorithms



CHAPTER 2: FUNCTION & RECURSION

1. FUNCTION
2. THE CONCEPT OF STACK
3. THE SEQUENCE OF EXECUTION DURING A FUNCTION CALL
4. Operator Scope
5. passing parameters type
6. RECURSION
8. Clones
9. Recursive rules
10. Function call stack with recursion
11. Recursive Vs. Iteration

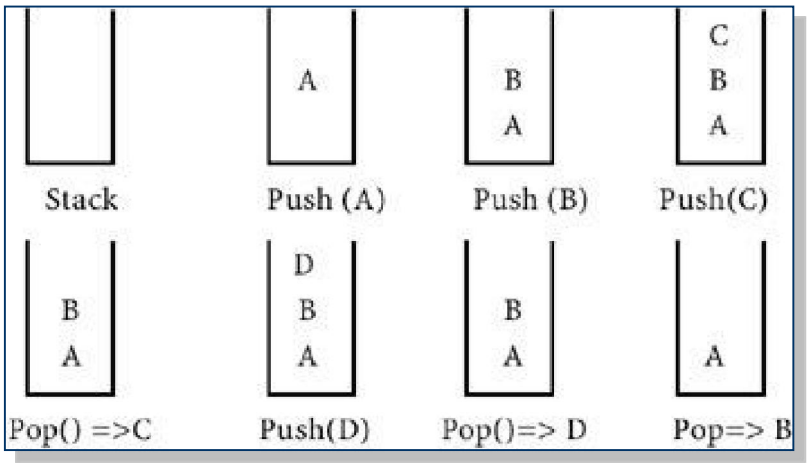
1. FUNCTION

- provide modularity to the software
- divide complex tasks into small manageable tasks
- avoid duplication of work

```
int add (int x, int y)
{
    int z;
    z = x + y;
    return z;
}
```

2.THE CONCEPT OF STACK

- تعتبر جزءًا من ذاكرة ROM وتستخدم لأغراض محددة وذلك لحفظ بعض القيم لاسترجاعها لاحقًاكما وتستخدم هذه الذاكرة بفاعلية عند استخدام البرامج الفرعية
- A *stack* is memory in which values are stored and retrieved in "last in first out" manner by using operations called *push* and *pop*.



3. THE SEQUENCE OF EXECUTION DURING A FUNCTION CALL

- When the function is called, the current execution is temporarily stopped and the control goes to the called function. After the call, the execution resumes from the point at which the execution is stopped.
- To get the exact point at which execution is resumed, the address of the next instruction is stored in the stack. When the function call completes, the address at the top of the stack is taken.

3. THE SEQUENCE OF EXECUTION DURING A FUNCTION CALL (cont)

- Functions or sub-programs are implemented using a stack.
- When a function is called, the address of the next instruction is pushed into the stack.
- When the function is finished, the address for execution is taken by using the pop operation.

Example :

- Result:?

```
void f1()
{
     printf ("f1-5 \n");
     printf ("f1-6 \n");
     f2 ( );
     printf ("f1-7 \n");
     printf ("f1-8 \n");
}
void f2 ()
{
     printf ("f2-9 \n");
     printf ("f2-10 \n");
}
main ( )
{
     printf ("1 \n");
     printf ("2 \n");
     f1();
     printf ("3 \n");
     printf ("4 \n");
}
```

4.Operator Scope انواع المتغيرات ومجال الرؤية لكل منها

When a variable can be resolved by using multiple references, the local definition is given more preference

```
50
7
10
```

```
#include<iostream>
using namespace std;
int i=10;//global variable
void f1()
{
    int i=50;//local variable for f1 function
    cout<<i<<endl;
}
void f2()
{
    cout<<i<<endl;
}
void main()
{
    int i=7;//local variable for main function
    f1();
    cout<<i<<endl;
    f2();
}
```

5.passing parameters type :

- *passing by value* التمرير بالقيمة
  - the value before and after the call remains the same
- *passing by reference* التمرير بالمرجعية (العنوان)
  - *changed value after the function completes*

passing by value

```
#include<iostream>
using namespace std;
int i=10;//global variable
int add_10 (int k)
{
    k+=10;
    return k;
}
void main()
{
    int i=1;//local variable for main function
    cout<<"before calling "<<i<<endl;
    add_10(i);
    cout<<"after calling "<<i<<endl;
}
```

```
C:\Windows\system32\cmd....
before calling 1
after calling 1
Press any key to continue . . .
```

passing by reference

```
#include<iostream>
using namespace std;
int i=10;//global variable
int add_10 (int *k)
{
    *k+=10;
    return *k;
}
void main()
{
    int i=1;//local variable for main function
    cout<<"before calling "<<i<<endl;
    add_10(&i);
    cout<<"after calling "<<i<<endl;
}
```

```
C:\Windows\system32\cm...
before calling 1
after calling 11
Press any key to continue . . .
```

Example :

Write C++ program to convert the input from inch to centimeter (using functions to convert and pass by reference to passing veriable) :

Hint: cm=inch * 2.54

6. Recursion

- Recursion is a function that calls itself
  - Repeats function block of code until a specified condition is met

```
int factorial() {
    ... factorial() ...
}
```

Q: Does using recursion usually make your code **faster**?
A: No

Q: Does using recursion usually use **less memory**?
A: No

Q: Then **why** use recursion?
A: It sometimes makes your code much **simpler!**

Example

```
void printInt( int k ) {
        if (k == 0)
        return;
        cout<< k ;
    printInt( k - 1 ); }
```

Assume:

-k is a positive integer

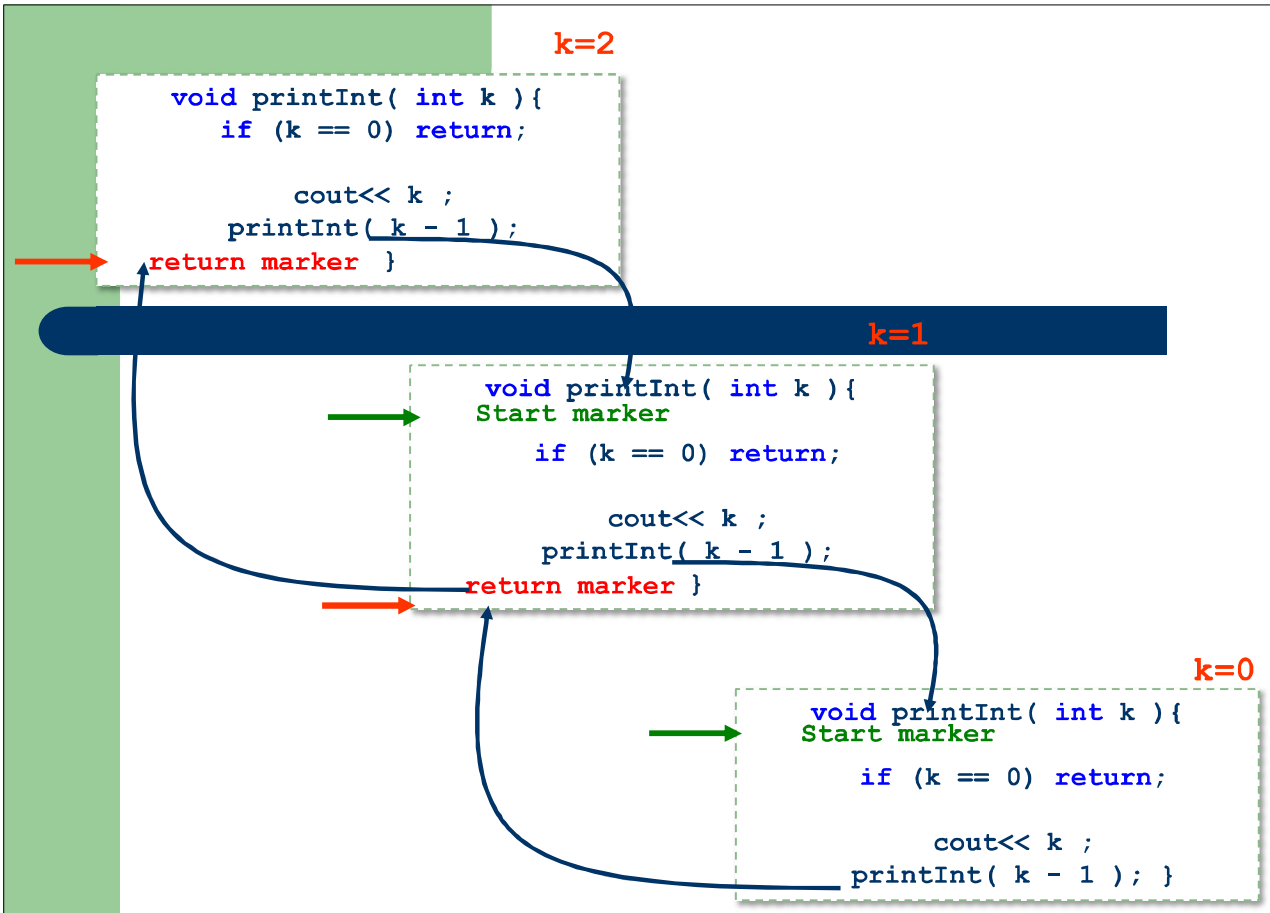
-the call: ***printInt(2);***

What are the values of K when this call is made ??

8. Clones

- When a recursive call is made, the method **clones itself**, making new copies of
  - Code
  - Local variables
  - Parameters

- The call ***printInt(2);*** will produce 3 clones (k=2,1,0)



9.Recursion Rules

-k is a positive integer

-the call: ***badPrint(2);***

```
void badPrint( int k ) {
        cout<< k ;
    badPrint( k + 1 ); }
```

How many clones do you think will be created ??

- Runtime Error: " StackOverflowError" and the program will stop
- Why ?? There is no code that prevents the recursive call from being made again and again and .... and eventually the program runs out of memory (to store all the clones)

  ➔ infinite recursion

Rule #1

- Every recursive function must have a **base case**
  - A condition under which no recursive call is made; to prevent infinite

```
void badPrint2( int k )
        {
    if (k < 0) return;

        cout<<k;
    badPrint2( k+1 );
        }
```

this version does have a base case, but the call *badPrint2(2)* will still cause an infinite recursion

-the call: ***badPrint2(2);***

What are the values of K when this call is made ??

Rule #2

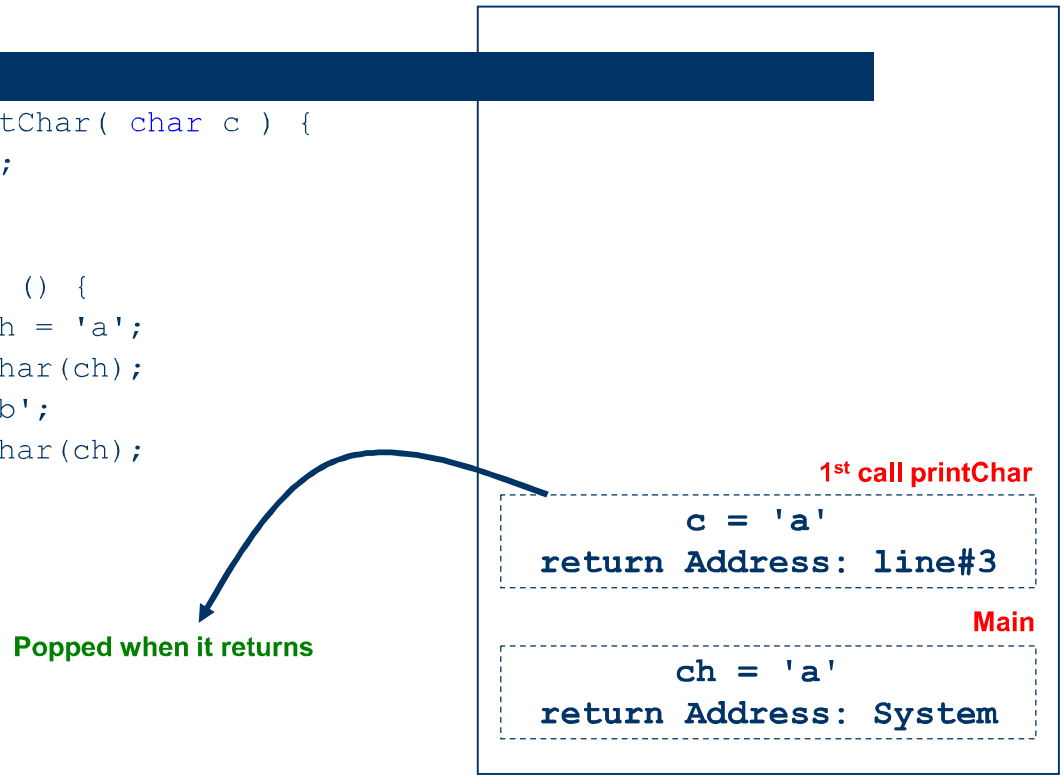
- Every recursive function must **make progress** toward the base case to prevent infinite recursion

```
void printInt( int k )
{
    if (k == 0) return;

    cout<< k ;
    printInt( k - 1 );
}
```

```
void badPrint2( int k )
{
    if (k < 0) return;

    cout<<k;
    badPrint2( k+1 );
}
```


10. Function Call Stack-No Recursion (1)

```
void printChar( char c ) {
  cout<<c;
}

void main () {
1.  char ch = 'a';
2.  printChar(ch);
3.  ch = 'b';
4.  printChar(ch);
}
```



Function Call Stack-No Recursion (2)

```
void printChar( char c ) {
  cout<<c;
}

void main () {
1.  char ch = 'a';
2.  printChar(ch);
3.  ch = 'b';
4.  printChar(ch)
5.  }
```

Popped when it returns

2nd call printChar

```
c = 'b'
return Address: line#5
```

Main

```
ch = 'b'
return Address: System
```


Function Call Stack-With Recursion (1)

```
1. void printInt( int k ) {
2.    if (k <= 0) return;
3.    cout<<k;
4.    printInt( k - 1 );
5. }

6. void main() {
7.    printInt(2);
8. }
```

Pushed after call to printInt from main

1st call printInt

```
k = 2
return Address: line#8
```

Main

```
return Address: System
```

Function Call Stack-With Recursion (2)

```
1. void printInt( int k ) {
2.     if (k <= 0) return;
3.     cout<<k;
4.     printInt( k - 1 );
5. }

6. void main() {
7.     printInt(2);
8. }
```

Pushed after first recursive call

2nd call printInt

k = 1
return Address: line#5

1st call printInt

k = 2
return Address: line#8

Main

return Address: System

Function Call Stack-With Recursion (3)

```
1. void printInt( int k ) {
2.     if (k <= 0) return;
3.     cout<<k;
4.     printInt( k - 1 );
5. }

6. void main() {
7.     printInt(2);
8. }
```

Pushed after second recursive call

3rd call printInt

k = 0
return Address: line#5

2nd call printInt

k = 1
return Address: line#5

1st call printInt

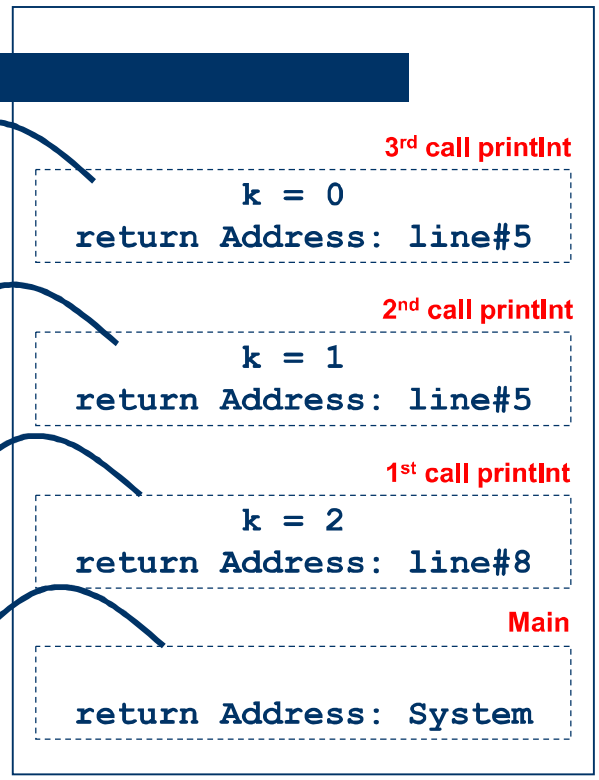
k = 2
return Address: line#8

Main

return Address: System

Function Call Stack-With Recursion (4)

```
1. void printInt( int k ) {
2.     if (k <= 0) return;
3.     cout<<k;
4.     printInt( k - 1 );
5. }

6. void main() {
7.     printInt(2);
8. }
```



11. Recursion Vs. Iteration

Q: Does using recursion usually make your code **faster**?

A: No, it's usually slower (due to the overhead of maintaining the stack)

Q: Does using recursion usually use **less memory**?

A: No, it usually uses **more** memory (for the stack)

Q: Then **why** use recursion?

A: It sometimes makes your code much **simpler!**

Example :Calculating Factorial by Iteration

- Factorial can be defined as follows
  - Factorial of 0 is 1
  - Factorial of N (for N>0) is N * N-1 * ... * 3 * 2 * 1

```
int factorial (int N) {
        int fact=1;
    if (N == 0) return 1;
for (int i=N; i>=1; i--)
          fact= fact*i;

          return fact;
              }
```


Example: Calculating Factorial by recursion

- Factorial can be defined as follows
  - Factorial of 0 is 1
  - Factorial of N (for N > 0) is N * factorial (N -1)

```
int factorial (int N)
          {
    if (N == 0) return 1;
else return N * factorial(N-1);
          }
```

Comparison

Recursion

```
int factorial (int N) {
   if (N == 0) return 1;
else return N * factorial(N-1);
}
```

Iterative

```
int factorial (int N) {
   int fact=1;
   if (N == 0) return 1;
for (int k=N; k>=1; k--)
   fact= fact*i;

   return fact;
}
```

- Recursive version
  - a little shorter
  - a little clearer (closer to the mathematical definition)
  - a little slower
  - more limited (it will fail, with a "stack overflow" error) for large values of N

Summary

- Use recursion for **clarity**, and (sometimes) for a reduction in the time needed to write and debug code, not for space savings or speed of execution
- Remember that every recursive function must have a **base case** (rule #1)
- Also remember that every recursive function must **make progress** towards its base case (rule #2)
- Anything that can be programmed using recursive functions can be programmed without recursion

Exercises (1) –

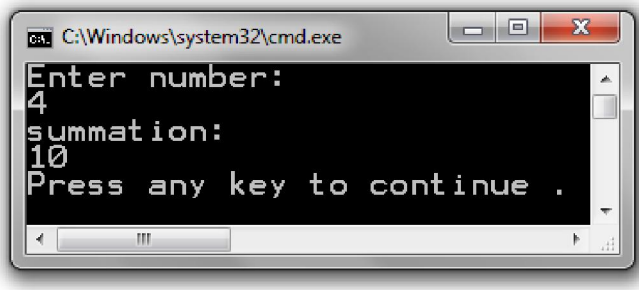
What does the following program do?

```
#include<iostream>
using namespace std;
int mystery (int a,int b)
{
    if (b==1) return a;
    else return a + mystery (a, b-1);
}
void main()
{
    int x,y;
    cout<<"enter tow integer:\n";
    cin>>x>>y;
    cout<<"the result is \n"<<mystery(x,y);
}
```


Exercises (2)

- Write a program to compute: S = 1 + 2 + 3 + …n using recursion.

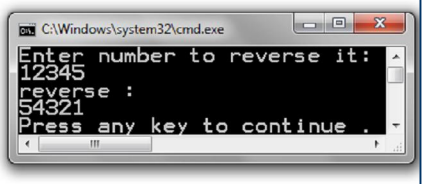
```
#include<iostream>
using namespace std;
int sum (int n){
    if(n==1)
        return 1;
    return n+sum(n-1);
}
void main(){
    int x;
    cout<<"Enter number:\n";
    cin>>x;
    cout<<"summation:\n"<<sum(x)<<endl;
 }
```



Excercises (3)

- Write a program to print a revert number Example: input n=12345. Print out: 54321.

```
#include<iostream>
using namespace std;
int y=0;
int reverse (int x){
    if(x%10==x)
    {
        y=y+x;
        return y;
    }
    else
    {
        y=y+(x%10);
        y=y*10;
        return reverse(x/10);
    }
}
void main(){
    int x;
    cout<<"Enter number to reverse it:\n";
    cin>>x;
    cout<<"reverse :\n"<<reverse(x)<<endl;
    }
```



Excercises (4)

- Write a program to print this number
  Example: input n=12345. Print out: 12345.

Excercises (5)

- Write a recursion function to find the sum of every number in a int number. Example: n=1980 => Sum=1+9+8+0=18.

```
#include<iostream>
using namespace std;
int y=0;
int sum_of_digits (int x){
    if(x%10==x)
    {
        y=y+x;
        return y;
    }
    else
    {
        y=y+(x%10);
        return sum_of_digits(x/10);
    }
}
void main(){
    int x;
    cout<<"Enter number to sum it digits:\n";
    cin>>x;
    cout<<"summation :\n"<<sum_of_digits(x)<<endl;
```

C:\Windows\system32\cmd.exe

```
Enter number to sum it digits:
1980
summation :
18
```

Excercises (5)

- Write a recursion function to calculate:
  - S=a[0]+a[1]+…a[n-1]
    - A: array of integer numbers

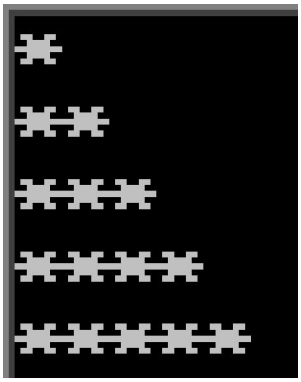
```
#include<iostream>
using namespace std;
int s=0;
int sum_of_array (int A[],int start,int end){

    if(start==end)
        return s;
    else
    {
        s+=A[start];
        return sum_of_array(A,start+1,end);
    }
}
void main(){
    const int size= 3;
    int A[size];
    cout<<"enter "<<size<<" element: ";
    for(int i=0;i<size;i++)
        cin>>A[i];
    cout<<"summation : "<<sum_of_array(A,0,size)<<endl;
```

C:\Windows\system32\cmd.exe

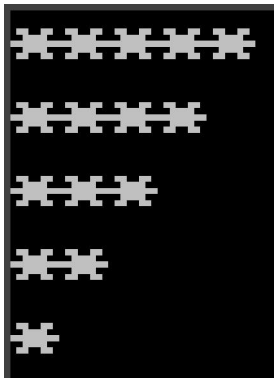
```
enter 3 element: 2 5 1
summation : 8
Press any key to continue . .
```

Excercises (6)

- Write a recursion function to Print triangles like bellow figure :

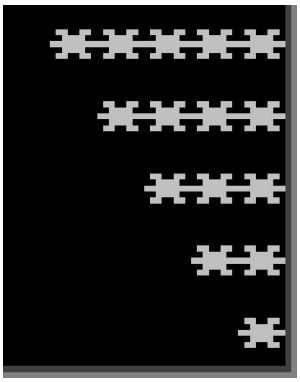


a

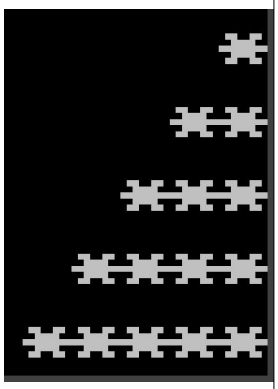


b

c



d



END